

TP1 CODESIGN

OpenCL Programming

Nour Ayari
Moatez Souilem
Mayar Chaabani
Ahmed Amin Chabbah
Eya Mhamdi

Introduction:

I. OpenCL Devices Specifications :

```
OpenCL Device Characteristics
=====

Platform: NVIDIA CUDA

-----

Device Name: NVIDIA GeForce RTX 3050 Laptop GPU
Device Type: ALL | GPU
Vendor: NVIDIA Corporation

Memory:
  Global: 4.00 GB
  Local: 48.0 KB
  Cache: 448.0 KB

Compute Units:
  SMs/CUs: 16

Work Items:
  Max Work-Group Size: 1024
  Max Work-Item Dimensions: 3
  Max Work-Item Sizes: [1024, 1024, 64]
-----
```

```
Platform: Intel(R) OpenCL Graphics
```

```
-----  
Device Name: Intel(R) UHD Graphics
```

```
Device Type: ALL | GPU
```

```
Vendor: Intel(R) Corporation
```

```
Memory:
```

```
Global: 10.71 GB
```

```
Local: 64.0 KB
```

```
Cache: 1920.0 KB
```

```
Compute Units:
```

```
SMs/CUs: 16
```

```
Work Items:
```

```
Max Work-Group Size: 512
```

```
Max Work-Item Dimensions: 3
```

```
Max Work-Item Sizes: [512, 512, 512]  
-----
```

```
Platform: Intel(R) OpenCL
```

```
-----  
Device Name: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz
```

```
Device Type: ALL | CPU
```

```
Vendor: Intel(R) Corporation
```

```
Memory:
```

```
Global: 23.73 GB
```

```
Local: 256.0 KB
```

```
Cache: 256.0 KB
```

```
Compute Units:
```

```
SMs/CUs: 12
```

```
Work Items:
```

```
Max Work-Group Size: 8192
```

```
Max Work-Item Dimensions: 3
```

```
Max Work-Item Sizes: [8192, 8192, 8192]  
-----
```

Devices comparison :

Device Type	NVIDIA RTX 3050 Laptop GPU	Intel UHD Graphics	Intel Core i5-11400H
Device Name	NVIDIA GeForce RTX 3050 Laptop GPU	Intel(R) UHD Graphics	11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz
Platform	NVIDIA CUDA	Intel OpenCL Graphics	Intel(R) OpenCL
Global Memory	4.00 GB	10.71 GB	23.73 GB
Local Memory	48 KB	64 KB	256 KB
Cache Size	448 KB	1920 KB	256 KB
Compute Units (SMs/CUs)	16	16	12 (CPU Cores)
Max Work-Group Size	1024	512	8192
Max Work-Item Sizes	[1024, 1024, 64]	[512, 512, 512]	[8192, 8192, 8192]
Memory Type	GDDR6	Shared System Memory	DDR4
Address Width	64-bit	64-bit	64-bit
Compute Capability	Ampere (8.6)	Gen11	x86-64

Part A: Matrix Multiplication Performance Analysis

A.1 CPU vs. OpenCL (CPU) Performance

Results:

a. Matrix multiplication: classic implementation on CPU (Sequential)

N=256 :

```
===== Sequential, matrix multiplication, order 256 on host CPU =====  
  
GFLOPS: 0.00  
END OF 1 Sequential Matrix Multiplication 256 * 256  
  
12.473533391952515 seconds at 2.6900502805121875 MFLOPS
```

N = 512 :

```
===== Sequential, matrix multiplication, order 512 on host CPU =====  
  
GFLOPS: 0.00  
END OF 1 Sequential Matrix Multiplication 512 * 512  
  
104.45680904388428 seconds at 2.569822479329473 MFLOPS
```

b. Matrix multiplication openCL implementations

N=256 , block size = 4

```
Starting 20 OpenCL Matrix Multiplications  
End of 20 Matrix Multiplications  
0.07923126220703125 seconds at 8469.99809553009 MFLOPS
```

N=512 , block size = 4

```
Starting 20 OpenCL Matrix Multiplications  
End of 20 Matrix Multiplications  
0.5958445072174072 seconds at 9010.25192809423 MFLOPS
```

Results Table

Matrix Size	Implementation	Time (s)	MFLOPS
N=256	CPU Sequential	12.473533391952515	2.6900502805121875
	OpenCL (CPU Coalesced)	0.03818678855895996	17573.843345424215
N=512	CPU Sequential	104.45680904388428	2.569822479329473
	OpenCL (CPU Coalesced)	0.30071496963500977	17853.148868898097

Performance Gap:

OpenCL on **CPU** is **~6,500x faster** than the **sequential CPU** implementation for both matrix sizes.

In fact :

- OpenCL leverages CPU threads/SIMD instructions to compute multiple elements simultaneously, while the sequential code uses a single thread.
- OpenCL compiles to optimized CPU instructions (e.g., AVX/SSE) for bulk floating-point operations.

Consistency in MFLOPS:

OpenCL MFLOPS remains stable (**~17,500–17,800**) for both **N=256** and **N=512**, suggesting strong **scalability**.

Sequential CPU MFLOPS slightly drops for **N=512**, likely due to cache misses and memory bottlenecks.

The sequential code's poor performance highlights why parallel frameworks like OpenCL are critical for computationally heavy tasks like matrix multiplication.

A.2 GPU Implementations :

1. Uncoalesced Implementation (Results in GFLOPS)

N = 2048

Device	4x4	8x8	16x16	32x32
Integrated GPU	66.28313339 454833	4.76696900406 9652	2.563909300118 626	=== Error for localsize = 32 ===
Dedicated GPU	108.5741468 3980135	151.204486004 41593	97.45032844322 357	57.142435517872 35
CPU	1.426479720 882878	1.18758269991 48792	9.550337134502 99	9.6439190598072 07

N =4096

Device	4x4	8x8	16x16	32x32
Integrated GPU	6.450061394 397512	4.75187127709 266	2.388052746625 2324	=== Error for localsize = 32 ===
Dedicated GPU	125.7073185 8815628	217.616605728 01504	116.0269864947 919	59.828661289673 01

N = 8192

Device	4x4	8x8	16x16	32x32
Integrated GPU				=== Error for localsize = 32 ===
Dedicated GPU	123.0815004 5270729	213.0561161850 3492	112.0280415082 9675	59.757615271886 665

Interpretations :

Integrated GPU :

- Performance degrades as work-group size increases (e.g., N=2048: 66.28 GFLOPS at 4x4 → 2.56 GFLOPS at 16x16).

Poor memory access patterns dominate over occupancy gains. Larger work-groups exacerbate uncoalesced memory access inefficiencies.

Uncoalesced accesses amplify this bottleneck, as each scattered memory request requires multiple transactions to fulfill.

- **Error for localsize = 32** : My Integrated GPU have strict constraints on the maximum number of threads per work-group which is 512 threads (table in 2.Devices Comparision) .
A 32x32 work-group requires 1024 threads, which exceeds this limit. That's why OpenCL kernels fails with CL_INVALID_WORK_GROUP_SIZE
- Larger matrices (N=4096) require more memory traffic. Uncoalesced access patterns overwhelm the integrated GPU's limited bandwidth, causing contention and stalling threads.
- **Missing Data for N=8192**: It takes too much time : Integrated GPUs lack parallel compute units to handle large matrices with uncoalesced accesses efficiently.

Hyper-sensitive to work-group size due to shared memory constraints.

Dedicated GPU :

- Peak at 8x8 work-group (e.g., N=2048: 151.20 GFLOPS at 8x8 → drops at 16x16).
- **Uncoalesced access wastes memory bandwidth; larger work-groups expose latency issues.**
- **For N=4096**, peak GFLOPS increases to 217.61 (8x8), but drops at 16x16 (116.02). This implies: Increased Parallelism Masking Inefficiency: Larger matrices provide more independent work, temporarily offsetting uncoalesced access penalties.
- Dedicated GPUs handle scaling better due to higher bandwidth, but efficiency still degrades.

Tolerates larger work-groups but still constrained by memory access patterns.

CPU :

Consistently Low GFLOPs due to:

- Lack of Parallelism: Even with multithreading, CPUs have fewer cores (e.g., 8–16 threads) compared to GPUs (1000s of threads).
- Clock Speed Dependency: Serial execution (limited SIMD) ties performance to clock speed (~3–5 GHz), which is dwarfed by GPU throughput.

2. Coalesced Implementation (Results in GFLOPS)

N = 2048

Device	4x4	8x8	16x16	32x32
Integrated GPU	8.145435817881929	13.953187842112944	15.819442376756713	=== Error for localsize = 32 ===
Dedicated GPU	140.98085931725684	348.05749099905125	436.52405983310204	433.1980112741529
CPU	1.286694395093549	1.0146394120519965	9.864365515710935	9.241699702529808

N =4096

Device	4x4	8x8	16x16	32x32
Integrated GPU	7.629939096 279505	12.3793666653 20273	16.80937913678 441	=== Error for localsize = 32 ===
Dedicated GPU	136.4171963 781084	342.417297060 38255	456.0704457293 7966	441.94569647328 125

N = 8192

Device	4x4	8x8	16x16	32x32
Integrated GPU				=== Error for localsize = 32 ===
Dedicated GPU	128.0352985 4866766	233.855975247 07284	446.8693977931 2763	432.02538953634 26

Integrated GPU :

Performance Improves with Larger Work-Groups:

For **N=2048**: 8.14 GFLOPS (4x4) → 15.81 GFLOPS (16x16).

For **N=4096**: 7.62 GFLOPS (4x4) → 16.80 GFLOPS (16x16)

Coalesced memory access aligns with the GPU's memory architecture, reducing redundant memory transactions. Larger work-groups improve occupancy (more threads hide memory latency).

Dedicated GPU Analysis

- Performance Peaks at 16x16 Work-Groups:

For **N=2048**: 436.52 GFLOPS (16x16) vs. 348.05 GFLOPS (8x8).

For **N=4096**: 456.07 GFLOPS (16x16) vs. 342.41 GFLOPS (8x8).

Larger work-groups better utilize the Dedicated GPU's massive parallelism (1000s of cores) and high memory bandwidth.

- **N=8192** Performance: Peak at 446.86 GFLOPS (16x16), slightly lower than N=4096.

Larger matrices stress memory subsystems, but coalesced access mitigates penalties.

CPU Analysis

Poor Performance Across All Work-Groups:

For **N=2048**: 1.28–9.86 GFLOPS (vs. Dedicated GPU's 140–436 GFLOPS).

CPUs lack the parallelism (cores/threads) and memory bandwidth to exploit coalesced access effectively.

Coalesced memory access optimizes global memory transactions by grouping requests into fewer, aligned operations. On dedicated GPUs, this reduces latency and maximizes bandwidth utilization, significantly boosting GFLOPS. For integrated GPUs, coalescing

alleviates bandwidth bottlenecks but cannot overcome the inherent limitations of shared system memory, resulting in modest performance gains.

3. Block_tiled Implementation : (Results in GFLOPS)

N = 2048

Device	4x4	8x8	16x16	32x32
Integrated GPU	17.74889675 6404698	34.3383852718 80565	32.96378550981 353	Error for localsize = 1
Dedicated GPU	Error for localsize = 1	390.301575925 98395	404.8762871612 0013	320.66635517733 89

N = 4096

Device	4x4	8x8	16x16	32x32
Integrated GPU	15.50910361 4917867	30.78038006711 2124	32.60945856421 7497	=== Error for localsize = 1 ===
Dedicated GPU	=== Error for localsize = 1 ===	415.560893514 17276	433.0155909021 50	338.68341843106 31

N = 8192

Device	4x4	8x8	16x16	32x32
Integrated GPU				
Dedicated GPU	=== Error for localsize = 1	408.69982280 89771	425.0414043516 187	339.05374533736 83

Integrated GPU :

Peak at 8x8 Work-Groups

- For N=2048: 34.33 GFLOPS (8x8) vs. 17.74 GFLOPS (4x4).
- For N=4096: 30.78 GFLOPS (8x8) vs. 15.50 GFLOPS (4x4).

Shared memory reuse improves performance, but larger tiles (16x16) exceed the Integrated GPU's limited shared memory capacity causing bank conflicts

Dedicated GPU Analysis

Peak at 16x16 Work-Groups:

- For N=2048: 404.87 GFLOPS (16x16) vs. 390.30 GFLOPS (8x8).
- For N=8192: 425.04 GFLOPS (16x16) vs. 339.05 GFLOPS (32x32).

Larger tiles maximize shared memory reuse, reducing global memory traffic. Dedicated GPUs have ample shared memory to support 16x16 tiles without register pressure.

Drop at 32x32 Work-Groups:

- Resource Limits: 32x32 tiles exhaust registers or shared memory, reducing occupancy (fewer active thread blocks).

Error at 4x4 Work-Groups:

- The block-tiled kernel likely requires a minimum tile size for shared memory logic to function. Smaller work-groups (4x4) break the kernel's assumptions.

A.2 Analysis: NVIDIA Dedicated GPU (N=8192)

1. UNCOALESCED (1x32) vs COALESCED (32x32)

Work-Group	Implementation	NVIDIA RTX 3050 (Est. GFLOPS)
32x1	Uncoalesced	39.44
32x32	Coalesced	432.0253895363426

Factor	Coalesced (32x32)	Uncoalesced (1x32)
Memory Access	Consecutive addresses (1 transaction/warp)	Scattered addresses (32 transactions/warp)
Work-Group Size	32x32 (1024 threads/block)	1x32 (32 threads/block)
Occupancy	High (~100%)	Low (~3%)
Memory Transactions	Reduced by 32x	Increased by 32x
Bandwidth Utilization	~90% of peak (efficient)	~10-20% of peak (inefficient)
GFLOPS	432.03	39.44

Coalesced access + large work-groups (32x32) exploit the GPU's memory hierarchy and parallelism, while uncoalesced + small work-groups (1x32) cripple both.

Performance gap $\approx 11\times$ (Memory efficiency + Occupancy).

2. COALESCED (1x32) vs UNCOALESCED (32x32)

Work-Group	Implementation	NVIDIA RTX 3050 (Est. GFLOPS)
32x1	Coalesced	40.40
32x32	Uncoalesced	59.757615271886665

Factor	Coalesced (1x32)	Uncoalesced (32x32)
Memory Access	Coalesced (sequential)	Uncoalesced (scattered)
Work-Group Size	1x32 (32 threads/block)	32x32 (1024 threads/block)
Occupancy	Low ($\sim 3\%$)	High ($\sim 100\%$)
Memory Transactions	1 transaction/warp (efficient)	32 transactions/warp (inefficient)
Bandwidth Utilization	$\sim 30-40\%$ of peak	$\sim 10-20\%$ of peak
GFLOPS	40.40	59.76
Bottleneck	Low occupancy (idle cores)	Memory inefficiency (wasted BW)

Performance Gap: $\sim 1.5\times$

- Large work-groups (32x32) are critical to fully utilize the GPU's parallel architecture.
- Coalesced (1x32) fails to exploit parallelism, despite efficient memory access.

Part B: Running the kernel on multiple OpenCL devices

1. Matrix splitting explanation :

N = 8192 , work-group size = 16 * 16
UNCOALSCED implementation

$$Speedup = \frac{Processing\ Time\ (Nvidia)}{Processing\ Time\ (3\ openCL\ devices)}$$

The matrix **A** is divided **row-wise** across the three OpenCL devices. Each device gets a certain number of rows from **A**, computes its corresponding portion of **C**, and the result is then combined into the final matrix.

Two test splits were used:

1. **[8192, 0, 0]**: All computation is done on the first device (reference time: ~11.0418 s)
2. **[7776, 144, 272]**: A more balanced split across 3 devices

The split **[7776, 144, 272]** was selected **empirically**, after testing various combinations. The main idea was that the DGPU is the best performer for matrix multiplications when N = 8192 as we can see in the PART A

However the iGPU was the least performer , as the device took too long to response

and the cpu wasn't giving good results in terms of time but better than iGPU

Assuming that , We assign:

- **Device 0**: 7776 rows (major workload, as it is fastest)
- **Device 1**: 144 rows (light load to avoid slowing down overall computation)
- **Device 2**: 272 rows (more than device 1, because it was slightly faster)

The split **[7776, 144, 272]** was chosen empirically to optimize performance: the majority of rows are handled by the fastest device (dGPU), while slower devices (iGPU, CPU) are assigned smaller parts.

2. OpenCL Host Code and obtained results :

```
# TP1partB_threaded.py - Parallel execution with multithreading
and split
from helper import *
from definitions import *
import numpy as np
import pyopencl as cl
import threading
from time import time

N = 8192
COUNT = 10

AVAL = 1.0
BVAL = 2.0

split_configs = [
    [8192, 0, 0],
    [7776, 144, 272],
    [7744, 160, 288],
    [7808, 128, 256],
]

best_speedup = 0
best_split = []
worst_speedup = float('inf')
worst_split = []

# Get OpenCL devices
platforms = cl.get_platforms()
devices = []
for platform in platforms:
    devices.extend(platform.get_devices())

selected_devices = [devices[0], devices[1], devices[2]]

# Load kernel
with open(r"C_elem_ij.cl") as f:
    kernel_source = f.read()

T_reference = None

for split in split_configs:
```

```

    if sum(split) != N:
        print(f"Skipping invalid split {split} (sum {sum(split)}
!= {N})")
        continue

    print(f"\n{'=' * 40}\nTesting split: {split}\n{'=' * 40}")

    try:
        h_A = np.full((N, N), AVAL, dtype=np.float32)
        h_B = np.full((N, N), BVAL, dtype=np.float32)
        h_C = np.zeros((N, N), dtype=np.float32)

        split_offset = [0, split[0], split[0] + split[1]]

        thread_times = [0.0, 0.0, 0.0]

        contexts, queues, buffers, kernels = [None]*3, [None]*3,
[None]*3, [None]*3

        for i in range(3):
            if split[i] == 0:
                continue

            ctx = cl.Context([selected_devices[i]])
            queue = cl.CommandQueue(ctx)

            A_part = h_A[split_offset[i]:split_offset[i] +
split[i], :].copy()
            mf = cl.mem_flags
            d_a = cl.Buffer(ctx, mf.READ_ONLY | mf.COPY_HOST_PTR,
hostbuf=A_part)
            d_b = cl.Buffer(ctx, mf.READ_ONLY | mf.COPY_HOST_PTR,
hostbuf=h_B)
            d_c = cl.Buffer(ctx, mf.WRITE_ONLY, size=split[i] * N
* 4)

            program = cl.Program(ctx, kernel_source).build()
            mmul = program.mmul
            mmul.set_scalar_arg_dtypes([np.int32, None, None,
None])
            mmul.set_args(np.int32(N), d_a, d_b, d_c)

            contexts[i] = ctx

```

```

        queues[i] = queue
        buffers[i] = (d_a, d_b, d_c)
        kernels[i] = mmul

    def run_kernel(i):
        if split[i] == 0:
            return
        global_size = (split[i], N)
        local_size = (16, 16)
        start_t = time()
        kernels[i](queues[i], global_size, local_size,
np.int32(N), *buffers[i])
        queues[i].finish()
        partial_C = np.empty((split[i], N), dtype=np.float32)
        cl.enqueue_copy(queues[i], partial_C,
buffers[i][2]).wait()
        h_C[split_offset[i]:split_offset[i] + split[i], :] =
partial_C
        thread_times[i] += time() - start_t

    # === Start COUNT timed executions ===
    start_total = time()
    for count_iter in range(COUNT):
        threads = []
        for i in range(3):
            if split[i] > 0:
                t = threading.Thread(target=run_kernel,
args=(i,))
                threads.append(t)
                t.start()

        for t in threads:
            t.join()
        total_time = time() - start_total

    if split == [8192, 0, 0]:
        T_reference = total_time

    speedup = T_reference / total_time if total_time and
T_reference else 0

    print(f"\nTotal time over {COUNT} runs: {total_time:.4f}
s - Speedup: {speedup:.2f}x")

```

```
        for i in range(3):
            if split[i] > 0:
                print(f" Device {i} - {split[i]} rows - Total
Time: {thread_times[i]:.4f} s")

            if speedup > best_speedup:
                best_speedup = speedup
                best_split = split.copy()
            if speedup < worst_speedup:
                worst_speedup = speedup
                worst_split = split.copy()

    except Exception as e:
        print(f"Error with split {split}: {str(e)}")
        continue

# Final summary
print("\n\n=== Final Results ===")
print(f"Best speedup: {best_speedup:.2f}x with split
{best_split}")
print(f"Worst speedup: {worst_speedup:.2f}x with split
{worst_split}")
```

This code parallelizes matrix multiplication (8192x8192) across 3 OpenCL devices by splitting rows of matrix A. Each split configuration (e.g., `[7808,128,256]`) assigns row chunks to devices. Threads launch kernels on each device: the first device handles the largest chunk, while others process smaller portions (<512 rows). Execution times are measured over 10 runs, comparing multi-device speedup against a single-device baseline (`[8192,0,0]`). Results track the best/worst splits based on speedup, leveraging OpenCL buffers and GPU threading for concurrent computation.

```
Total time over 1 runs: 10.7608 s - Speedup: 1.03x
Device 0 - 7776 rows - Total Time: 10.7488 s
Device 1 - 144 rows - Total Time: 8.0060 s
Device 2 - 272 rows - Total Time: 9.5079 s
```